

```
import pandas as pd
# basic data exploration
# 1. Load Titanic-Dataset.csv
df = pd.read_csv("/content/Titanic-Dataset.csv")
print(df)
```

```
   PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3
..          ...         ...     ...
886          887         0       2
887          888         1       1
888          889         0       3
889          890         1       1
890          891         0       3
```

```
   Name                               Sex  Age  SibSp \
0  Braund, Mr. Owen Harris             male  22.0    1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0    1
2  Heikkinen, Miss. Laina              female  26.0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)   female  35.0    1
4  Allen, Mr. William Henry            male   35.0    0
..          ...          ...     ...
886  Montvila, Rev. Juozas              male   27.0    0
887  Graham, Miss. Margaret Edith         female  19.0    0
888  Johnston, Miss. Catherine Helen "Carrie"    female   NaN    1
889  Behr, Mr. Karl Howell               male   26.0    0
890  Dooley, Mr. Patrick                 male   32.0    0
```

```
   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0   PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0  113803  53.1000  C123      S
4      0  373450   8.0500   NaN      S
..    ...     ...     ...     ...
886     0  211536  13.0000   NaN      S
887     0  112053  30.0000  B42      S
888     2   W./C. 6607  23.4500   NaN      S
889     0  111369  30.0000  C148      C
890     0  370376   7.7500   NaN      Q
```

[891 rows x 12 columns]

```
# 1. Create a Series from the Age column.
age_series = df["Age"]
print(age_series)
# 2. Create a Series from the Fare column with passenger names as the index.
fare_series = pd.Series(df["Fare"].values, index=df["Name"])
print(fare_series)
# 3. Load the Titanic dataset into a DataFrame.
df = pd.read_csv("/content/Titanic-Dataset.csv")
print(df)
# 4. Create a new DataFrame containing only Name , Age , and Sex .
new_df = df[["Name", "Age", "Sex"]]
print(new_df)
```

```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
...
886   27.0
887   19.0
888    NaN
889   26.0
890   32.0
Name: Age, Length: 891, dtype: float64
Name
Braund, Mr. Owen Harris              7.2500
Cumings, Mrs. John Bradley (Florence Briggs Thayer)  71.2833
Heikkinen, Miss. Laina                7.9250
Futrelle, Mrs. Jacques Heath (Lily May Peel)  53.1000
Allen, Mr. William Henry              8.0500
...
Montvila, Rev. Juozas                13.0000
```

```
Graham, Miss. Margaret Edith      30.0000
Johnston, Miss. Catherine Helen "Carrie" 23.4500
Behr, Mr. Karl Howell             30.0000
Dooley, Mr. Patrick               7.7500
Length: 891, dtype: float64
```

```
PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
..          ...      ...    ...
886         887         0       2
887         888         1       1
888         889         0       3
889         890         1       1
890         891         0       3
```

```
                Name      Sex  Age  SibSp  \
0      Braund, Mr. Owen Harris  male  22.0    1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0    1
2      Heikkinen, Miss. Laina  female  26.0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0    1
4      Allen, Mr. William Henry  male  35.0    0
..          ...      ...    ...    ...
886      Montvila, Rev. Juozas  male  27.0    0
887      Graham, Miss. Margaret Edith  female  19.0    0
888  Johnston, Miss. Catherine Helen "Carrie"  female   NaN    1
889      Behr, Mr. Karl Howell  male  26.0    0
890      Dooley, Mr. Patrick  male  32.0    0
```

```
Parch      Ticket      Fare  Cabin  Embarked
0         0      A/5 21171   7.2500   NaN      S
1         0      PC 17599  71.2833   C85      C
2         0  STON/O2. 3101282   7.9250   NaN      S
3         0      113803  53.1000  C123      S
4         0      373450   8.0500   NaN      S
..      ...      ...      ...      ...
```

```
# 5. Display the first 10 rows.
df.head(10)
# 6. Display the last 10 rows.
df.tail(10)
# 7. Display 5 random rows.
df.sample(10)
# 8. Print all column names.
df.columns
# 9. Find the shape of the dataset.
df.shape
# 10. Display dataset information using info() .
df.info()
# 11. Generate descriptive statistics using describe() .
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
# 12. Create FamilySize column
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["FamilySize"]
```

FamilySize	
0	2
1	2
2	1
3	2
4	1
...	...
886	1
887	1
888	4
889	1
890	1

891 rows × 1 columns

dtype: int64

```
# 13. Create Fare_per_Person column
df["Fare_per_Person"] = df["Fare"] / df["FamilySize"]
df["Fare_per_Person"]
```

```
# 14. Convert all passenger names to uppercase
df["Name"] = df["Name"].str.upper()
df["Name"]
```

```
Survived
0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
...      ...
886     NaN
887     NaN
888     NaN
889     NaN
890     NaN
891 rows × 1 columns
```

dtype: object

```
# 16. Create Age_Group column
def age_group(age):
    if age < 18:
        return "Child"
    elif age <= 60:
        return "Adult"
    else:
        return "Senior"

df["Age_Group"] = df["Age"].apply(age_group)
df["Age_Group"]

# 17. Map Survived values
df["Survived"] = df["Survived"].map({0: "No", 1: "Yes"})
df["Survived"]
```

```
Survived
0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
...      ...
886     NaN
887     NaN
888     NaN
889     NaN
890     NaN
891 rows × 1 columns
```

dtype: object

```
# 18. Select only the Name column.
df["Name"]
```

Name	
0	BRAUND, MR. OWEN HARRIS
1	CUMINGS, MRS. JOHN BRADLEY (FLORENCE BRIGGS TH...
2	HEIKKINEN, MISS. LAINA
3	FUTRELLE, MRS. JACQUES HEATH (LILY MAY PEEL)
4	ALLEN, MR. WILLIAM HENRY
...	...
886	MONTVILA, REV. JUOZAS
887	GRAHAM, MISS. MARGARET EDITH
888	JOHNSTON, MISS. CATHERINE HELEN "CARRIE"
889	BEHR, MR. KARL HOWELL
890	DOOLEY, MR. PATRICK

891 rows × 1 columns

dtype: object

```
# 19. Select Name , Sex , and Age .  
df[["Name", "Sex", "Age"]]
```

	Name	Sex	Age
0	BRAUND, MR. OWEN HARRIS	male	22.0
1	CUMINGS, MRS. JOHN BRADLEY (FLORENCE BRIGGS TH...	female	38.0
2	HEIKKINEN, MISS. LAINA	female	26.0
3	FUTRELLE, MRS. JACQUES HEATH (LILY MAY PEEL)	female	35.0
4	ALLEN, MR. WILLIAM HENRY	male	35.0
...	...	...	...
886	MONTVILA, REV. JUOZAS	male	27.0
887	GRAHAM, MISS. MARGARET EDITH	female	19.0
888	JOHNSTON, MISS. CATHERINE HELEN "CARRIE"	female	NaN
889	BEHR, MR. KARL HOWELL	male	26.0
890	DOOLEY, MR. PATRICK	male	32.0

891 rows × 3 columns

```
# 20. Select the first 20 rows using iloc .  
df.iloc[:20]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	FamilySize	Far
0	1	NaN	3	BRAUND, MR. OWEN HARRIS	male	22.0	1	0	A/5 21171	7.2500	NaN	S	2	
1	2	NaN	1	CUMINGS, MRS. JOHN BRADLEY (FLORENCE BRIGGS TH...	female	38.0	1	0	PC 17599	71.2833	C85	C	2	
2	3	NaN	3	HEIKKINEN, MISS. LAINA	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	1	
3	4	NaN	1	FUTRELLE, MRS. JACQUES HEATH (LILY MAY PEEL)	female	35.0	1	0	113803	53.1000	C123	S	2	
4	5	NaN	3	ALLEN, MR. WILLIAM HENRY	male	35.0	0	0	373450	8.0500	NaN	S	1	
5	6	NaN	3	MORAN, MR. JAMES	male	NaN	0	0	330877	8.4583	NaN	Q	1	
6	7	NaN	1	MCCARTHY, MR. TIMOTHY J	male	54.0	0	0	17463	51.8625	E46	S	1	
7	8	NaN	3	PALSSON, MASTER. GOSTA LEONARD	male	2.0	3	1	349909	21.0750	NaN	S	5	
8	9	NaN	3	JOHNSON, MRS. OSCAR W (ELISABETH VILHELMINA BERG)	female	27.0	0	2	347742	11.1333	NaN	S	3	
9	10	NaN	2	NASSER, MRS. NICHOLAS (ADELE ACHEM)	female	14.0	1	0	237736	30.0708	NaN	C	2	
10	11	NaN	3	SANDSTROM, MISS. MARGUERITE RUT	female	4.0	1	1	PP 9549	16.7000	G6	S	3	
11	12	NaN	1	BONNELL, MISS. ELIZABETH	female	58.0	0	0	113783	26.5500	C103	S	1	
12	13	NaN	3	SAUNDERCOCK, MR. WILLIAM HENRY	male	20.0	0	0	A/5. 2151	8.0500	NaN	S	1	
13	14	NaN	3	ANDERSSON, MR. ANDERS JOHAN	male	39.0	1	5	347082	31.2750	NaN	S	7	
14	15	NaN	3	VESTROM, MISS. HULDA AMANDA ADOLFINA	female	14.0	0	0	350406	7.8542	NaN	S	1	
15	16	NaN	2	HEWLETT, MRS. (MARY D KINGCOME)	female	55.0	0	0	248706	16.0000	NaN	S	1	
16	17	NaN	3	RICE, MASTER. EUGENE	male	2.0	4	1	382652	29.1250	NaN	Q	6	
17	18	NaN	2	WILLIAMS, MR. CHARLES EUGENE	male	NaN	0	0	244373	13.0000	NaN	S	1	
18	19	NaN	3	VANDER PLANKE, MRS. JULIUS (EMELIA MARIA VANDE...	female	31.0	1	0	345763	18.0000	NaN	S	2	
19	20	NaN	3	MASSELMANI, MRS. FATIMA	female	NaN	0	0	2649	7.2250	NaN	C	1	

```
# 21. Select rows 10 to 30 and columns 2 to 6 using iloc .  
df.iloc[10:31, 2:7]
```

	Pclass	Name	Sex	Age	SibSp
10	3	SANDSTROM, MISS. MARGUERITE RUT	female	4.0	1
11	1	BONNELL, MISS. ELIZABETH	female	58.0	0
12	3	SAUNDERCOCK, MR. WILLIAM HENRY	male	20.0	0
13	3	ANDERSSON, MR. ANDERS JOHAN	male	39.0	1
14	3	VESTROM, MISS. HULDA AMANDA ADOLFINA	female	14.0	0
15	2	HEWLETT, MRS. (MARY D KINGCOME)	female	55.0	0
16	3	RICE, MASTER. EUGENE	male	2.0	4
17	2	WILLIAMS, MR. CHARLES EUGENE	male	NaN	0
18	3	VANDER PLANKE, MRS. JULIUS (EMELIA MARIA VANDE...	female	31.0	1
19	3	MASSELMANI, MRS. FATIMA	female	NaN	0
20	2	FYNNEY, MR. JOSEPH J	male	35.0	0
21	2	BEESLEY, MR. LAWRENCE	male	34.0	0
22	3	MCGOWAN, MISS. ANNA "ANNIE"	female	15.0	0
23	1	SLOPER, MR. WILLIAM THOMPSON	male	28.0	0
24	3	PALSSON, MISS. TORBORG DANIRA	female	8.0	3
25	3	ASPLUND, MRS. CARL OSCAR (SELMA AUGUSTA EMILIA...	female	38.0	1
26	3	EMIR, MR. FARRED CHEHAB	male	NaN	0
27	1	FORTUNE, MR. CHARLES ALEXANDER	male	19.0	3
28	3	O'DWYER, MISS. ELLEN "NELLIE"	female	NaN	0
29	3	TODOROFF, MR. LALIO	male	NaN	0
30	1	URUCHURTU, DON. MANUEL E	male	40.0	0

```
# 22. Find all passengers older than 50
df[df["Age"] > 50]

# 23. Find all female passengers
df[df["Sex"] == "female"]

# 24. Find passengers who survived
df[df["Survived"] == 1]

# 25. Find passengers whose fare is greater than 100
df[df["Fare"] > 100]
```


PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
27	28	0	1	Fortune, Mr. Charles Alexander	male	19.00	3	2	19950	263.0000	C23 C25 C27	S
31	32	1	1	Spencer, Mrs. William Augustus (Marie Eugenie)	female	NaN	1	0	PC 17569	146.5208	B78	C
88	89	1	1	Fortune, Miss. Mabel Helen	female	23.00	3	2	19950	263.0000	C23 C25 C27	S
118	119	0	1	Baxter, Mr. Quigg Edmond	male	24.00	0	1	PC 17558	247.5208	B58 B60	C
195	196	1	1	Lurette, Miss. Elise	female	58.00	0	0	PC 17569	146.5208	B80	C
215	216	1	1	Newell, Miss. Madeleine	female	31.00	1	0	35273	113.2750	D36	C
258	259	1	1	Ward, Miss. Anna	female	35.00	0	0	PC 17755	512.3292	NaN	C
268	269	1	1	Graham, Mrs. William Thompson (Edith Jenkins)	female	58.00	0	1	PC 17582	153.4625	C125	S
269	270	1	1	Bissette, Miss. Amelia	female	35.00	0	0	PC 17760	135.6333	C99	S
297	298	0	1	Allison, Miss. Helen Loraine	female	2.00	1	2	113781	151.5500	C22 C26	S
299	300	1	1	Baxter, Mrs. James (Helene DeLauniere Chaput)	female	50.00	0	1	PC 17558	247.5208	B58 B60	C
305	306	1	1	Allison, Master. Hudson Trevor	male	0.92	1	2	113781	151.5500	C22 C26	S
306	307	1	1	Fleming, Miss. Margaret	female	NaN	0	0	17421	110.8833	NaN	C
307	308	1	1	Penasco y Castellana, Mrs. Victor de Satode (M...	female	17.00	1	0	PC 17758	108.9000	C65	C

Start coding or generate with AI.

```
# 26. Find passengers aged between 20 and 40
df[(df["Age"] >= 20) & (df["Age"] <= 40)]

# 27. Find passengers who embarked from S or C
df[df["Embarked"].isin(["S", "C"])]

# 28. Find passengers who did not survive
df[df["Survived"] == 0]

# 29. Find male passengers in first class
df[(df["Sex"] == "male") & (df["Pclass"] == 1)]
```

Titanic - Passenger Data												
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
281	322	1	1	Forster, Mrs. William Elizabeth	female	24.00	0	3	1199360	2535.0000	C23 C25 C27	S
27	28	0	1	Fortune, Mr. Charles Alexander	male	19.0	3	2	19950	263.0000	C23 C25 C27	S
307	338	0	1	Widener, Mr. Harry Elkins	male	40.00	0	0	PC 17601	271.2000	NaN	C
34	35	0	1	Meyer, Mr. Edgar Joseph	male	28.0	1	0	PC 17604	82.1708	NaN	C
390	391	1	1	Carter, Mr. William...	male	36.00	1	2	113760	120.0000	B96 B98	S
839	840	1	1	Marechal, Mr. Pierre	male	NaN	0	0	11774	29.7000	C47	C
853	854	1	1	newell, Miss. Marjorie	female	23.00	0	0	35213	113.2100	D36	C
857	858	1	1	Daly, Mr. Peter Denis	male	51.0	0	0	113055	26.5500	E17	S
865	866	1	1	Carter, Miss. Lucille Polly	female	44.00	1	2	113760	120.0000	B96 B98	C
867	868	0	1	Roebing, Mr. Washington Augustus II	male	31.0	0	0	PC 17590	50.4958	A24	S

```
# 30. Sort passengers by age (ascending by default)
df.sort_values("Age")
```

```
# 31. Sort passengers by fare in descending order
df.sort_values("Fare", ascending=False)
```

```
# 32. Sort by Pclass and then by Fare
df.sort_values(["Pclass", "Fare"])
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
263	0	1	Douglas, Mr. Walter	male	50.00	1	0	11064250	0.00	C86	C
264	0	1	Harrison, Mr. William	male	40.0	0	0	112059	0.00	B94	S
550	0	1	Thayer, Mr. John Borland	male	17.00	0	0	1742111200	8830.00	C70	S
806	0	1	Parr, Mr. William Henry Mars	male	NaN	0	2	1742111200	8830.00	C70	S
807	0	1	Andrews, Mr. Thomas Jr	male	39.0	0	0	112050	0.00	A36	S
815	0	1	Robbins, Mr. Victor	male	NaN	0	0	17757	227.3250	NaN	C
816	0	1	Fry, Mr. Richard	male	NaN	0	0	112058	0.00	B102	S
822	0	1	Reuchlin, Jonkheer. John George	male	38.0	0	0	19972	0.00	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
201	0	3	Shutes, Miss. Elizabeth	...	...	...	...	PC	...	...	...
202	0	3	Sage, Mr. Frederick	male	NaN	8	2	CA. 2343	69.55	NaN	S
659	0	1	Webster	male	58.00	0	2	35213	113.2150	D48	C
324	0	3	Sage, Mr. George John Jr	male	NaN	8	2	CA. 2343	69.55	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
792	0	3	Frauenthal, Dr. Henry	...	...	...	...	PC	...	...	...
793	0	3	Sage, Miss. Stella Anna	female	NaN	8	2	CA. 2343	69.55	NaN	S
679	1	1	Drake Martinez	male	36.00	0	1	17755	512.3292	B55	C
...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	Medill, Miss. Georgette	...	...	...	...	...	...	...	...

```
# 33. Split dataset into two parts and concatenate vertically
top_100 = df.head(100)
bottom_100 = df.tail(100)
```

```
vertical_concat = pd.concat([top_100, bottom_100], axis=0)
print(vertical_concat)
```

```
# 34. Create two DataFrames with different columns and concatenate horizontally
df1 = df[["Name", "Age"]]
df2 = df[["Sex", "Fare"]]
```

```
horizontal_concat = pd.concat([df1, df2], axis=1)
print(horizontal_concat)
```

	PassengerId	Survived	Pclass	\		Name	Sex	Age	SibSp	\
0	1	0	3		0	Braund, Mr. Owen Harris	male	22.0	1	
1	2	1	1		1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	3	1	3		2	Heikkinen, Miss. Laina	female	26.0	0	
3	4	1	1		3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	5	0	3		4	Allen, Mr. William Henry	male	35.0	0	
..	...	...	...		..	...	...	...	...	
886	887	0	2		886	Montvila, Rev. Juozas	male	27.0	0	
887	888	1	1		887	Graham, Miss. Margaret Edith	female	19.0	0	
888	889	0	3		888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	
889	890	1	1		889	Behr, Mr. Karl Howell	male	26.0	0	
890	891	0	3		890	Dooley, Mr. Patrick	male	32.0	0	
	Parch	Ticket	Fare	Cabin	Embarked					
0	0	A/5 21171	7.2500	NaN	S					
1	0	PC 17599	71.2833	C85	C					
2	0	STON/O2. 3101282	7.9250	NaN	S					
3	0	113803	53.1000	C123	S					
4	0	373450	8.0500	NaN	S					
..	...	...	...	...	...					

```

886      0      211536  13.0000  NaN      S
887      0      112053  30.0000  B42      S
888      2      W./C. 6607  23.4500  NaN      S
889      0      111369  30.0000  C148      C
890      0      370376   7.7500  NaN      Q

```

[200 rows x 12 columns]

	Name	Age	Sex	Fare
0	Braund, Mr. Owen Harris	22.0	male	7.2500
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	female	71.2833
2	Heikkinen, Miss. Laina	26.0	female	7.9250
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	female	53.1000
4	Allen, Mr. William Henry	35.0	male	8.0500
..	...	...	...	...
886	Montvila, Rev. Juozas	27.0	male	13.0000
887	Graham, Miss. Margaret Edith	19.0	female	30.0000
888	Johnston, Miss. Catherine Helen "Carrie"	NaN	female	23.4500
889	Behr, Mr. Karl Howell	26.0	male	30.0000
890	Dooley, Mr. Patrick	32.0	male	7.7500

[891 rows x 4 columns]

```

# 37. Fill missing Age values with mean age
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Age"]

```

	Age
0	22.000000
1	38.000000
2	26.000000
3	35.000000
4	35.000000
...	...
886	27.000000
887	19.000000
888	29.699118
889	26.000000
890	32.000000

891 rows x 1 columns

dtype: float64

```

# 38. Fill missing Embarked values with mode
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df["Embarked"]

```

```

# 39. Fill missing Cabin values with "Unknown"
df["Cabin"] = df["Cabin"].fillna("Unknown")
df["Cabin"]

```

Cabin

0 Unknown

```
# 42. Find the maximum fare
print(df["Fare"].max())
```

```
# 43. Find the minimum age
print(df["Age"].min())
```

```
# 44. Find the average age
print(df["Age"].mean())
```

```
51273292 B42
0.42
.88895Unknown:382
```

```
# 45. Find the median fare
print(df["Fare"].median())
```

```
# 46. Find the standard deviation of fare
print(df["Fare"].std())
```

```
# 47. Find unique embarkation points
print(df["Embarked"].unique())
```

```
# 48. Count passengers by gender
print(df["Sex"].value_counts())
```

```
# 49. Find the most common passenger class
print(df["Pclass"].mode()[0])
```

```
14.4542
49.693428597180905
['S' 'C' 'Q']
Sex
male      577
female    314
Name: count, dtype: int64
3
```

```
# 50. Calculate average age by gender
df.groupby("Sex")["Age"].mean()
```

```
# 51. Calculate average fare by passenger class
df.groupby("Pclass")["Fare"].mean()
```

```
# 52. Count passengers in each embarkation port
df.groupby("Embarked").size()
```

0

Embarked

C 168

Q 77

S 646

dtype: int64

```
# 53. Calculate survival rate by gender
df.groupby("Sex")["Survived"].mean()
```

```
# 54. Find maximum fare in each class
```